Schritt 36 - GraphQL Aliases, konkret wie Schritt 35

Die zentrale Frage:

> Gehört ein Alias zu GraphQL oder zu Rails? Befindet er sich im Client-Request?

Die Antwort:

Ein Alias ist GraphQL-Syntax und wird direkt im GraphQL-Dokument geschrieben, das der Client an /graphql sendet. Rails benötigt keine eigene Klasse und keinen eigenen Resolver für einen Alias.

1. Was ist ein Alias?

Ein Alias erlaubt dem Client:

```
1. einen Key in der Response umzubenennen
2. dasselbe Field mehrfach mit unterschiedlichen Arguments aufzurufen
```

Syntax:

```
aliasName: fieldName
```

Beispiel:

```
query GetProject($id: ID!) {
  project(id: $id) {
    projectName: name
  }
}
```

Das Rails-Schema enthält weiterhin:

```
field :name,
      String,
      null: false
```

Der Client erhält:

```
{
  "data": {
    "project": {
      "projectName": "CRM"
    }
  }
}
```

Nicht:

```
{
  "name": "CRM"
}
```

2. Alias gehört zu GraphQL, nicht zu Rails

Der Client schreibt:

```
projectName: name
```

Rails braucht nicht:

```
field :project_name
```

Und auch nicht:

```
def project_name
end
```

Das Schema benötigt nur das ursprüngliche Field:

```
field :name,
      String,
      null: false
```

Die GraphQL Engine versteht:

```
projectName
→ Response Key

name
→ tatsächliches Field im Schema
```

3. Wichtigster Anwendungsfall: dasselbe Field, unterschiedliche Arguments

Angenommen, du möchtest:

```
TODO Tasks
IN_PROGRESS Tasks
DONE Tasks
```

Wenn du Folgendes schreibst:

```
query GetTaskGroups($projectId: ID!) {
  tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
  ) {
    nodes {
      id
    }
  }

  tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
  ) {
    nodes {
      id
    }
  }
}
```

entsteht ein GraphQL-Konflikt, weil beide denselben Response Key haben:

```
tasks
```

aber unterschiedliche Arguments verwenden.

Aliases lösen das:

```
query GetTaskGroups($projectId: ID!) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      id
      title
    }
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) {
    nodes {
      id
      title
    }
  }
```

```
}
```

4. Response eines Alias

Response:

```
{
  "data": {
    "todoTasks": {
      "nodes": [
        {
          "id": "1",
          "title": "Build API"
        }
      ]
    },

    "doneTasks": {
      "nodes": [
        {
          "id": "2",
          "title": "Setup Devise"
        }
      ]
    }
  }
}
```

Der Client verwendet:

```
response.data.data.todoTasks.nodes
```

und:

```
response.data.data.doneTasks.nodes
```

5. Wo steht der Alias?

Der Alias steht direkt im GraphQL-Dokument:

```
todoTasks: tasks(...)
```

Der Client sendet dieses Dokument mit Axios:

```
const query = `
  query GetTaskGroups($projectId: ID!) {
    todoTasks: tasks(
      projectId: $projectId
      filter: {
        status: [TODO]
      }
      first: 10
    ) {
      nodes {
        id
        title
      }
    }

    doneTasks: tasks(
      projectId: $projectId
      filter: {
        status: [DONE]
      }
      first: 10
    ) {
      nodes {
        id
        title
      }
    }
  }
`;
```

6. Vollständiger Axios Request

```
import axios from "axios";

const query = `
  query GetTaskGroups($projectId: ID!) {
    todoTasks: tasks(
      projectId: $projectId
      filter: {
        status: [TODO]
      }
      first: 10
    ) {
      nodes {
        id
        title
        status
        priority
      }
    }

    inProgressTasks: tasks(
      projectId: $projectId
      filter: {
        status: [IN_PROGRESS]
      }
      first: 10
    ) {
      nodes {
        id
        title
        status
        priority
      }
    }

    doneTasks: tasks(
      projectId: $projectId
      filter: {
        status: [DONE]
      }
      first: 10
    ) {
      nodes {
        id
        title
        status
        priority
      }
    }
  }
`;

const response = await axios.post(
  "http://localhost:3000/graphql",
  {
    query,

    variables: {
      projectId: "10"
    },

    operationName: "GetTaskGroups"
  },
  {
    withCredentials: true
  }
);
```

7. Wie rendert der Client die Daten?

```
const todoTasks =
  response.data.data.todoTasks.nodes;

const inProgressTasks =
  response.data.data.inProgressTasks.nodes;
```

```
const doneTasks =
  response.data.data.doneTasks.nodes;
```

React:

```
function TaskBoard({ data }) {
  const todoTasks =
    data.todoTasks.nodes;

  const inProgressTasks =
    data.inProgressTasks.nodes;

  const doneTasks =
    data.doneTasks.nodes;

  return (
    <div>
      <section>
        <h2>TODO</h2>

        {todoTasks.map((task) => (
          <div key={task.id}>
            {task.title}
          </div>
        ))}
      </section>

      <section>
        <h2>IN PROGRESS</h2>

        {inProgressTasks.map((task) => (
          <div key={task.id}>
            {task.title}
          </div>
        ))}
      </section>

      <section>
        <h2>DONE</h2>

        {doneTasks.map((task) => (
          <div key={task.id}>
            {task.title}
          </div>
        ))}
      </section>
    </div>
  );
}
```

8. Alias + Fragment

Schritt 35 lässt sich sehr gut mit Schritt 36 kombinieren.

Fragment:

```
fragment TaskSummaryFields on Task {
  id
  title
  status
  priority
  dueDate
}
```

Query:

```
query GetTaskBoard($projectId: ID!) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
```

```
  }

  inProgressTasks: tasks(
    projectId: $projectId
    filter: {
      status: [IN_PROGRESS]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }

  doneTasks: tasks(
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

9. Alias + Variables

Ein Alias ersetzt Variables nicht.

Beispiel:

```
query CompareProjects(
  $firstId: ID!
  $secondId: ID!
) {
  firstProject: project(
    id: $firstId
  ) {
    id
    name
  }

  secondProject: project(
    id: $secondId
  ) {
    id
    name
  }
}
```

Variables:

```
{
  "firstId": "10",
  "secondId": "20"
}
```

Hier gilt:

```
firstProject
secondProject
→ Alias

$firstId
$secondId
→ Variables
```

10. Alias + Mutation

Aliases können auch in Mutations verwendet werden.

Beispiel:

```
mutation CreateTwoTasks(
```

```
  $first: CreateTaskInput!
  $second: CreateTaskInput!
) {
  firstTask: createTask(
    input: $first
  ) {
    task {
      id
      title
    }

    errors
  }

  secondTask: createTask(
    input: $second
  ) {
    task {
      id
      title
    }

    errors
  }
}
```

Variables:

```
{
  "first": {
    "projectId": "10",
    "attributes": {
      "title": "Backend"
    }
  },

  "second": {
    "projectId": "10",
    "attributes": {
      "title": "Frontend"
    }
  }
}
```

Response:

```
{
  "data": {
    "firstTask": {
      "task": {
        "id": "1",
        "title": "Backend"
      },
      "errors": []
    },

    "secondTask": {
      "task": {
        "id": "2",
        "title": "Frontend"
      },
      "errors": []
    }
  }
}
```

11. Vorsicht bei mehreren Mutation-Aliases

GraphQL erlaubt:

```
firstTask: createTask(...)
secondTask: createTask(...)
```

aber du darfst das nicht automatisch als Datenbank-Transaktion betrachten.

Wenn:

```
firstTask
→ success

secondTask
→ fail
```

bedeutet das nicht, dass firstTask automatisch zurückgerollt wird.

Wenn die Business-Logik verlangt:

```
10 Tasks erstellen
→ entweder alle erfolgreich
→ oder alles rollback
```

solltest du eine eigene Mutation bzw. einen Service erstellen:

```
createTasksBatch
```

und verwenden:

```
ActiveRecord::Base.transaction
```

12. Reduziert ein Alias die Anzahl der Requests?

Es kann die Anzahl der HTTP Requests reduzieren.

Statt:

```
Request 1
→ TODO tasks

Request 2
→ DONE tasks
```

kannst du einen GraphQL Request senden:

```
Request 1
→ todoTasks
→ doneTasks
```

Das bedeutet aber nicht automatisch weniger SQL.

13. Reduziert ein Alias SQL?

Nicht automatisch.

Die Query:

```
todoTasks: tasks(...)
doneTasks: tasks(...)
```

kann zu Folgendem führen:

```
todoTasks
→ TasksResolver
→ SQL query 1

doneTasks
→ TasksResolver
→ SQL query 2
```

Ein Alias hilft bei:

```
1 HTTP GraphQL request
```

Das bedeutet nicht:

```
1 SQL query
```

14. Alias und Dashboard

Du könntest zum Beispiel Folgendes schreiben:

```
query GetTaskDashboard(
  $projectId: ID!
) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
```

```
      status: [TODO]
    }
    first: 5
  ) {
    nodes {
      id
      title
    }
  }

  overdueTasks: tasks(
    projectId: $projectId
    filter: {
      overdue: true
    }
    first: 5
  ) {
    nodes {
      id
      title
    }
  }
}
```

Das ist sinnvoll, wenn du nur wenige kleine Task-Gruppen brauchst.

Wenn ein Dashboard aber Folgendes benötigt:

```
totalProjects
todoCount
doneCount
overdueCount
10 Arten von Statistiken
```

ist ein dediziertes Field:

```
dashboard {
  ...
}
```

oft besser, weil das Backend SQL-Aggregationen effizienter durchführen kann.

15. Verändert ein Alias das Schema?

Nein.

Beispiel Schema:

```
field :name,
      String,
      null: false
```

Client:

```
projectTitle: name
```

Das Schema bleibt:

```
name
```

Die Introspection zeigt weiterhin:

```
name: String!
```

Der Alias gehört nur zur Operation des Clients.

16. Kann ein Alias für Security verwendet werden?

Nein.

Du solltest niemals:

```
if alias_name == "adminTasks"
```

zur Autorisierung verwenden.

Der Client kann selbst schreiben:

```
totallySafe: deleteProject(...)
```

Ein Alias hat keinen Security-Wert.

Authorization muss weiterhin auf Folgendem basieren:

```
current_user
Policy
Policy::Scope
```

17. Unterschied Alias und Fragment

```
Alias
→ Response Key umbenennen
→ oder dasselbe Field mehrfach aufrufen

Fragment
→ Selection Set wiederverwenden
```

Beispiel:

```
todoTasks: tasks {
  nodes {
    ...TaskFields
  }
}
```

Hier:

```
todoTasks
→ Alias

...TaskFields
→ Fragment
```

18. Unterschied Alias und Operation Name

```
query GetTaskBoard {
```

GetTaskBoard ist:

```
Operation Name
```

Dagegen:

```
todoTasks: tasks
```

todoTasks ist:

```
Alias
```

Das sind zwei völlig unterschiedliche Dinge.

19. Unterschied Alias und Variable

```
todoTasks: tasks(
  projectId: $projectId
)
```

Hier:

```
todoTasks
→ Alias

$projectId
→ Variable
```

Der Alias verändert den Namen in der Response.

Die Variable trägt einen dynamischen Wert.

20. Diagramm zu Schritt 36

```
CLIENT
│
├── Operation
│   GetTaskBoard
│
├── Variables
```

```
│   projectId
│
├── Alias
│   todoTasks
│   doneTasks
│
└── Fragment
    TaskFields
        │
        ▼
GraphQL document
        │
        ▼
POST /graphql
        │
        ▼
Rails GraphQL Schema
        │
        ▼
tasks field
        │
        ├── todoTasks
        │    └── TasksResolver
        │
        └── doneTasks
             └── TasksResolver
        │
        ▼
GraphQL Response
        │
        ├── todoTasks
        └── doneTasks
```

Merksätze zu Schritt 36

```
Alias
→ gehört zu GraphQL

Wo wird es geschrieben?
→ im GraphQL-Dokument des Clients

Braucht Rails eine Alias-Klasse?
→ nein

Wozu dient es?
→ Response Key umbenennen
→ dasselbe Field mehrfach mit unterschiedlichen Arguments aufrufen

In Query verwendbar?
→ ja

In Mutation verwendbar?
→ ja

Reduziert es SQL?
→ nicht automatisch

Für Authorization verwenden?
→ nein
```

Praktischstes Beispiel für dieses Projekt:

```
query GetTaskBoard($projectId: ID!) {
  todoTasks: tasks(
    projectId: $projectId
    filter: {
      status: [TODO]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }

  doneTasks: tasks(
```

```
    projectId: $projectId
    filter: {
      status: [DONE]
    }
    first: 10
  ) {
    nodes {
      ...TaskSummaryFields
    }
  }
}
```

Dabei gilt:

```
GetTaskBoard
→ Operation Name

$projectId
→ Variable

todoTasks / doneTasks
→ Aliases

...TaskSummaryFields
→ Fragment
```

Genau so werden Schritt 33–36 zu einem realistischen GraphQL-Client-Request kombiniert.